

Universidad de Costa Rica  
Facultad de Ingeniería  
Escuela de Ingeniería Eléctrica  
IE0435 - Inteligencia Artificial Aplicada a la Ingeniería Eléctrica  
I Ciclo 2026

## Proyecto 1 - Informe Final

Sebastián Rojas Gutiérrez  
C26797

12 de mayo del 2026

# Índice

|                                                                                        |           |
|----------------------------------------------------------------------------------------|-----------|
| <b>Índice</b>                                                                          | <b>II</b> |
| <b>1. Repositorio de trabajo</b>                                                       | <b>1</b>  |
| <b>2. Descripción del problema</b>                                                     | <b>1</b>  |
| <b>3. Conjunto de datos</b>                                                            | <b>1</b>  |
| 3.1. Recolección y organización . . . . .                                              | 1         |
| 3.2. Partición del dataset . . . . .                                                   | 1         |
| 3.3. Limitaciones del dataset . . . . .                                                | 2         |
| <b>4. Desarrollo de la solución</b>                                                    | <b>2</b>  |
| 4.1. Preprocesamiento de imágenes . . . . .                                            | 2         |
| 4.2. Generación del dataset . . . . .                                                  | 3         |
| 4.3. Selección del modelo . . . . .                                                    | 3         |
| 4.4. Arquitectura del modelo SVM . . . . .                                             | 4         |
| 4.5. Entrenamiento y optimización del umbral . . . . .                                 | 5         |
| <b>5. Diseño final</b>                                                                 | <b>6</b>  |
| <b>6. Herramientas de evaluación e inferencia</b>                                      | <b>6</b>  |
| 6.1. Evaluación sobre CSV ( <code>test.py</code> ) . . . . .                           | 6         |
| 6.2. Inferencia sobre carpeta de imágenes ( <code>predict_folder.py</code> ) . . . . . | 6         |
| <b>7. Dificultades</b>                                                                 | <b>7</b>  |
| <b>8. Resultados</b>                                                                   | <b>7</b>  |
| 8.1. Evaluación sobre el conjunto de validación interno . . . . .                      | 7         |
| 8.2. Evaluación sobre datos de prueba independientes . . . . .                         | 7         |
| 8.3. Análisis de generalización . . . . .                                              | 10        |
| <b>9. Conclusiones</b>                                                                 | <b>10</b> |
| <b>10. Modelos fundacionales utilizados</b>                                            | <b>11</b> |
| <b>Referencias</b>                                                                     | <b>11</b> |

## 1. Repositorio de trabajo

Todos los scripts, datasets y documentación correspondientes a la elaboración de este proyecto pueden ser consultados en este [repositorio](#).

## 2. Descripción del problema

El proyecto consiste en desarrollar un sistema de clasificación automática de imágenes orientado a la inspección de calidad en una línea de producción simulada. En este escenario, se utiliza una hoja blanca como superficie de fondo sobre la cual pueden aparecer distintos objetos que representan posibles contaminaciones. Los **granos de arroz** se definen como el contaminante de interés, por lo que toda imagen que contenga al menos un grano de arroz constituye un **ejemplo positivo** (imagen contaminada, etiqueta 1). Por otro lado, las imágenes que no contienen arroz —aunque puedan incluir otros objetos como aros o clips, o no presentar ningún objeto— se clasifican como **ejemplos negativos** (imagen limpia, etiqueta 0).

El objetivo central del proyecto consiste en diseñar, entrenar y evaluar un modelo de aprendizaje supervisado capaz de distinguir de forma automática entre imágenes contaminadas y limpias, a partir de una representación matricial binaria de cada imagen. El sistema debe ser robusto ante variaciones de iluminación y posición de los objetos, y aspira a alcanzar un desempeño mínimo del 90 % de *recall* en ambas clases.

Para ello, el flujo de trabajo abarca desde la adquisición y preprocesamiento de las imágenes, pasando por la construcción del conjunto de datos estructurado, hasta el entrenamiento, optimización y evaluación del modelo seleccionado.

## 3. Conjunto de datos

### 3.1. Recolección y organización

Las imágenes del dataset fueron recopiladas a partir del aporte de múltiples compañeros del curso, con el fin de obtener una base de datos variada que reflejara distintas condiciones de captura: dispositivos de cámara heterogéneos, variaciones de iluminación, diferentes ángulos y distancias de toma. Esta diversidad enriquece el dataset, aunque también introduce variabilidad que supone un reto adicional para el modelo.

Las imágenes fueron organizadas manualmente en dos carpetas según su etiqueta:

- **positive/**: imágenes que contienen al menos un grano de arroz (etiqueta 1).
- **negative/**: imágenes sin contaminación visible (etiqueta 0).

### 3.2. Partición del dataset

El dataset completo se dividió en dos particiones mediante el script `generar_dataset.py`, utilizando una semilla fija (`SEMILLA = 42`) para garantizar la reproducibilidad exacta de los resultados:

Tabla 1: Distribución del dataset por partición y clase

| Partición                                | Positivos | Negativos | Total |
|------------------------------------------|-----------|-----------|-------|
| Entrenamiento ( <code>train.csv</code> ) | 100       | 100       | 200   |
| Prueba ( <code>test.csv</code> )         | 30        | 30        | 60    |
| <b>Total</b>                             | 130       | 130       | 260   |

El balance exacto 50/50 entre clases fue impuesto de forma deliberada para evitar sesgos por desbalanceo durante el entrenamiento. No obstante, esta distribución artificial puede no reflejar la proporción real de

eventos en un entorno de producción. Cada fila del CSV resultante contiene 16 384 valores enteros (0 o 1) correspondientes a los píxeles de la imagen binarizada, seguidos de la etiqueta de clase.

### 3.3. Limitaciones del dataset

El dataset presenta varias limitaciones que condicionan el desempeño máximo del modelo:

- **Tamaño reducido:** 200 muestras de entrenamiento producen alta varianza en los modelos. El rendimiento es sensible a la partición aleatoria utilizada.
- **Variabilidad de captura no controlada:** iluminación, dispositivo de cámara, ángulo y distancia no fueron estandarizados entre sesiones.
- **Único tipo de contaminante:** el dataset solo contempla granos de arroz. Otros contaminantes no están representados.
- **Fondo asumido blanco:** el pipeline de binarización asume fondo predominantemente blanco; superficies de otro color pueden producir binarizaciones incorrectas.

## 4. Desarrollo de la solución

La solución se estructura en cuatro componentes principales: el preprocesamiento de imágenes, la generación del dataset en formato tabular, la selección y entrenamiento del modelo, y la optimización del umbral de clasificación.

### 4.1. Preprocesamiento de imágenes

Las imágenes fueron capturadas sobre una superficie blanca para facilitar la segmentación por contraste. El script `generar_dataset.py` aplica el siguiente pipeline de preprocesamiento a cada imagen de forma uniforme:

1. **Redimensionado:** la imagen se escala a  $128 \times 128$  píxeles mediante interpolación `INTER_AREA`, que minimiza el aliasing en operaciones de reducción de resolución.
2. **Conversión a escala de grises:** se elimina la dependencia de la información de color, reduciendo la imagen a un único canal de intensidad.
3. **Suavizado gaussiano:** se aplica un filtro Gaussiano de  $5 \times 5$  para atenuar el ruido de alta frecuencia antes de la umbralización.
4. **Umbralización adaptativa gaussiana:** en lugar de un umbral global, se calcula un umbral local por bloques de  $31 \times 31$  píxeles con constante  $C = 5$ . Este enfoque es significativamente más robusto ante variaciones de iluminación dentro de la imagen que el método de Otsu utilizado en versiones anteriores del script.
5. **Binarización:** los píxeles se mapean a 0 (objeto presente) o 1 (fondo blanco).
6. **Aplanado:** la matriz  $128 \times 128$  se convierte en un vector de 16 384 valores, listo para ser procesado por el clasificador.

```

1 img = cv2.resize(img, (IMG_SIZE, IMG_SIZE),
2                   interpolation=cv2.INTER_AREA)
3 img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
4 img = cv2.GaussianBlur(img, (5, 5), 0)
5 binarizada = cv2.adaptiveThreshold(
6     img, 255,
7     cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
8     cv2.THRESH_BINARY,
9     ADAPT_BLOCK, ADAPT_C
10 )
11 vector = (binarizada // 255).flatten().tolist()

```

Listing 1: Pipeline de preprocesamiento aplicado a cada imagen

Los parámetros globales que controlan el preprocesamiento se definen como constantes al inicio del script, lo que facilita su ajuste y reproducción:

```

1 IMG_SIZE          = 128
2 USAR_UMBRAL_ADAPT = True
3 ADAPT_BLOCK       = 31
4 ADAPT_C           = 5
5 SEMILLA           = 42
6 TRAIN_POR_CLASE   = 100
7 TEST_POR_CLASE    = 30

```

Listing 2: Parámetros globales de preprocesamiento y partición

## 4.2. Generación del dataset

Una vez preprocesada cada imagen, el vector resultante junto con su etiqueta se agregan a una lista de filas que posteriormente se serializa como archivo CSV mediante la librería **pandas**:

```

1 columnas = [f"p{i}" for i in range(IMG_SIZE * IMG_SIZE)] + ["etiqueta"]
2 df = pd.DataFrame(filas_dataset, columns=columnas)
3 df.to_csv("dataset.csv", index=False)

```

Listing 3: Construcción del DataFrame y exportación a CSV

La división train/test se realiza con balance exacto por clase y mezcla aleatoria controlada:

```

1 df_train, df_test = train_test_split(
2     df, test_size=TEST_POR_CLASE * 2,
3     random_state=SEMILLA, stratify=df["etiqueta"]
4 )

```

Adicionalmente, el script guarda matrices de verificación en la carpeta **verificacion/** para permitir la inspección visual de la binarización de cada imagen y detectar posibles errores en el preprocesamiento.

## 4.3. Selección del modelo

Durante el desarrollo del proyecto se evaluaron siete algoritmos de clasificación supervisada con el objetivo de identificar el que ofreciera el mejor equilibrio entre *recall* positivo y negativo. Todos los modelos fueron entrenados con el mismo conjunto de datos y evaluados sobre el mismo split de validación interno (20 % de **train.csv**, estratificado). Los modelos explorados y sus resultados aproximados sobre el conjunto de validación se resumen en la Tabla 2.

Tabla 2: Comparación de modelos evaluados sobre el conjunto de prueba

| Modelo            | Accuracy      |
|-------------------|---------------|
| Árbol de decisión | < 0,60        |
| Naive Bayes       | < 0,60        |
| KNN               | < 0,60        |
| Random Forest     | 0.6667        |
| Híbrido SVM→RF    | 0.70          |
| Híbrido RF→SVM    | 0.75          |
| <b>SVM (RBF)</b>  | <b>0.7333</b> |

La **máquina de soporte vectorial con kernel RBF** fue seleccionada como modelo final a pesar de que el híbrido RF→SVM registró una exactitud ligeramente superior (75 %). La razón principal es que dicha diferencia de 1.67 puntos porcentuales, sobre un conjunto de prueba de solo 60 muestras, no es estadísticamente significativa y puede atribuirse a varianza de muestreo. En contraste, la SVM presenta ventajas concretas: su pipeline es más simple (tres etapas frente a un ensamble de dos modelos entrenados con *stacking*), sus hiperparámetros son fijos y documentados, y su proceso de optimización del umbral es completamente reproducible. Los modelos basados en árbol de decisión, Naive Bayes y KNN quedaron descartados desde etapas tempranas al no superar el 60 % de exactitud, mientras que Random Forest (66.67 %) y el híbrido SVM→RF (70 %) mostraron un desempeño inferior al de la SVM sin las ventajas de simplicidad que esta ofrece.

#### 4.4. Arquitectura del modelo SVM

El modelo implementado en `SVM/svm.py` consiste en un pipeline secuencial de tres etapas encadenadas mediante la clase `Pipeline` de `scikit-learn`:

1. **Estandarización** (`StandardScaler`): centra cada característica en media cero con varianza unitaria. Esta etapa es indispensable para el correcto funcionamiento del kernel RBF, que es sensible a la escala de las características.
2. **Reducción de dimensionalidad** (`PCA`): se retienen 30 componentes principales, reduciendo el espacio original de 16 384 dimensiones a 30. Esta reducción elimina correlaciones entre píxeles adyacentes, mejora la generalización y reduce significativamente el tiempo de entrenamiento.
3. **Clasificador SVM** (`SVC`): kernel RBF con parámetro de regularización  $C = 20$  y ancho de kernel  $\gamma = 0,003$ . Se habilita la estimación de probabilidades mediante el método de Platt y se utiliza `class_weight='balanced'` para compensar cualquier desbalanceo residual en los lotes de entrenamiento.

```

1 N_COMPONENTS = 30
2 C             = 20.0
3 GAMMA        = 0.003
4
5 pipeline = Pipeline([
6     ("scaler", StandardScaler()),
7     ("pca", PCA(n_components=N_COMPONENTS, random_state=42)),
8     ("svm", SVC(kernel="rbf", C=C, gamma=GAMMA,
9                 probability=True,
10                class_weight="balanced",
11                random_state=42)),

```

12 ])

Listing 4: Definición del pipeline SVM en svm.py

Los hiperparámetros  $C$ ,  $\gamma$  y el número de componentes PCA fueron determinados mediante búsqueda sistemática sobre el conjunto de validación interno, priorizando el equilibrio entre las métricas de ambas clases.

#### 4.5. Entrenamiento y optimización del umbral

El conjunto de entrenamiento (`train.csv`, 200 muestras) se dividió internamente en 80% para ajuste del modelo ( $n = 160$ ) y 20% para validación del umbral ( $n = 40$ ), mediante `train_test_split` con estratificación por clase y semilla fija para garantizar la reproducibilidad:

```
1 X_train, X_val, y_train, y_val = train_test_split(
2     X, y, test_size=0.2, random_state=42, stratify=y
3 )
4 pipeline.fit(X_train, y_train)
```

Una vez ajustado el pipeline, se estimaron las probabilidades de clase positiva sobre el conjunto de validación y se procedió a buscar el umbral de clasificación óptimo. En lugar de emplear el umbral estándar de 0.5, se exploró el intervalo  $[0,05, 0,95]$  en 181 pasos uniformes para maximizar el *geometric recall*:

$$\text{geometric recall} = \sqrt{\text{recall}_{\text{neg}} \times \text{recall}_{\text{pos}}} \quad (1)$$

Esta métrica penaliza de manera equitativa el sesgo hacia cualquiera de las dos clases: si el modelo favorece fuertemente una clase, el producto de los dos *recalls* disminuye, lo que incentiva un punto de operación equilibrado.

```
1 best_geo, best_thresh = 0.0, 0.5
2 y_proba = pipeline.predict_proba(X_val)[: , 1]
3
4 for t in np.linspace(0.05, 0.95, 181):
5     pred_t = (y_proba >= t).astype(int)
6     geo     = geometric_recall(y_val, pred_t)
7     if geo > best_geo:
8         best_geo, best_thresh = geo, t
```

Listing 5: Búsqueda exhaustiva del umbral óptimo

El pipeline entrenado y el umbral óptimo se encapsulan juntos en un objeto `ThresholdedRF`, un *wrapper* que almacena el umbral para el proceso de inferencia, y se serializan en `svm_model.joblib` mediante `joblib`:

```
1 model = ThresholdedRF(pipeline=pipeline, threshold=best_thresh)
2 joblib.dump(model, "svm_model.joblib")
```

Para este caso específico, el modelo entrenado puede encontrarse en el repositorio como `c26797_sebastian.rojas.joblib`. Para reproducir el entrenamiento desde cero con los mismos resultados, basta con ejecutar:

```
1 # Requiere: train.csv en la carpeta raiz del proyecto
2 # Dependencias: pip install -r requirements.txt
3 python SVM/svm.py
4 # Parametros fijos: N_COMPONENTS=30, C=20.0, GAMMA=0.003
5 # Semillas fijas: PCA(random_state=42), SVC(random_state=42)
6 #                 train_test_split(random_state=42, stratify=y)
7 # Salida: svm_model.joblib
```

Listing 6: Comando de entrenamiento reproducible

## 5. Diseño final

El sistema desarrollado sigue una arquitectura de tipo *pipeline* de extremo a extremo, estructurada en tres módulos claramente diferenciados: generación del dataset, entrenamiento del modelo e inferencia sobre nuevas imágenes.

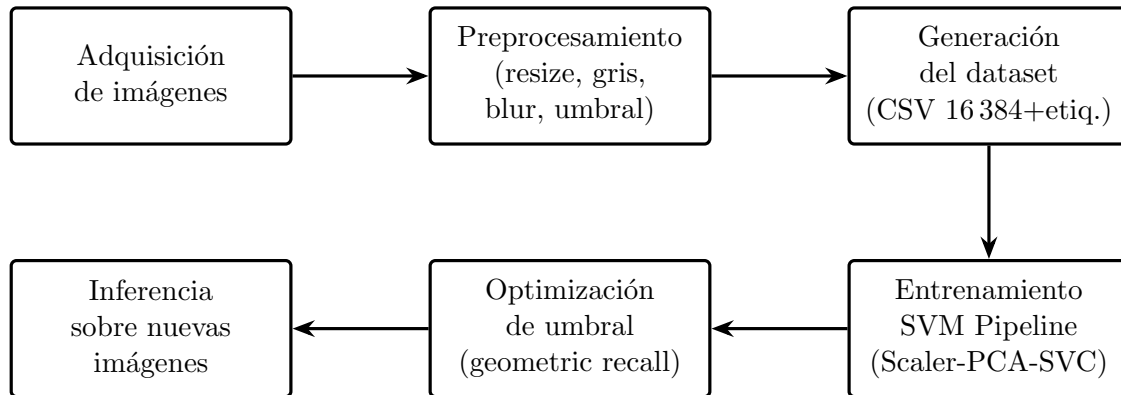


Figura 1: Arquitectura general del sistema de detección de contaminación

La modularidad del diseño garantiza que cada etapa pueda ajustarse de forma independiente sin afectar las demás, siempre que se respeten las interfaces de entrada y salida definidas. Por ejemplo, modificar los parámetros de preprocesamiento en `generar_dataset.py` requiere regenerar el dataset y reentrenar el modelo, pero no implica cambios en el código de inferencia.

El uso de técnicas como el umbral adaptativo gaussiano y la reducción de dimensionalidad mediante PCA permite mejorar la robustez del sistema frente a variaciones en las condiciones de captura, que constituyeron la principal fuente de dificultad durante el desarrollo.

## 6. Herramientas de evaluación e inferencia

### 6.1. Evaluación sobre CSV (`test.py`)

El script `test.py` permite evaluar el modelo SVM de forma directa sobre un archivo CSV generado con `generar_dataset.py`. Al ejecutarse, el usuario puede indicar si el CSV incluye etiquetas reales, en cuyo caso se calculan métricas completas (exactitud, precisión, *recall* por clase, F1 y matriz de confusión) y se generan gráficas comparativas. Este módulo fue el empleado para la evaluación formal sobre `test.csv`.

El script es autocontenido: importa y carga el modelo, aplica la predicción con el umbral ya almacenado en el objeto `ThresholdedRF`, y exporta los resultados a archivos CSV con marca temporal.

### 6.2. Inferencia sobre carpeta de imágenes (`predict_folder.py`)

El script `predict_folder.py` simula el escenario de producción real: recibe como entrada una carpeta con imágenes sin procesar, aplica internamente el mismo pipeline de preprocesamiento de `generar_dataset.py`, garantizando coherencia con los datos de entrenamiento, y obtiene predicciones del modelo SVM directamente desde las imágenes originales.

El script detecta automáticamente el modo de operación según la estructura de la carpeta de entrada:

- **Modo predicción:** si la carpeta contiene imágenes en su raíz (sin subdirectorios `positive//negative/`), se generan únicamente predicciones sin cálculo de métricas.

- **Modo evaluación:** si la carpeta contiene los subdirectorios `positive/` y `negative/`, las etiquetas reales se infieren de la estructura de carpetas, sin pasarlas al modelo, y se calculan todas las métricas de desempeño: exactitud, precisión, *recall* por clase y F1.

En ambos casos, los resultados se guardan automáticamente en un archivo CSV con marca temporal (`predicciones_SVM_<timestamp>.csv`).

## 7. Dificultades

Durante el desarrollo del proyecto se presentaron diversas dificultades tanto en la etapa de recolección y preprocesamiento de datos como en el entrenamiento y validación del modelo.

La principal fuente de dificultad fue la variabilidad en las condiciones de captura de las imágenes. Al haberse recopilado fotografías de distintos compañeros con diferentes dispositivos, las imágenes presentan variaciones significativas en iluminación, ángulo de toma y distancia al objeto. Si bien el umbral adaptativo gaussiano mitiga parcialmente estas diferencias dentro de cada imagen, no elimina completamente el efecto producido por cámaras con distribuciones de píxeles distintas, lo que introduce inconsistencias en las matrices binarizadas.

La reducida cantidad de datos disponibles (200 muestras de entrenamiento) constituyó otra limitación importante. Con un dataset de este tamaño, los modelos presentan alta varianza entre ejecuciones y tienen dificultades para generalizar a condiciones no representadas durante el entrenamiento. Este factor limitó el desempeño máximo alcanzable independientemente del algoritmo empleado.

La evaluación y selección entre múltiples modelos también presentó un reto metodológico. Se exploraron siete configuraciones distintas, incluyendo ensambles y métodos híbridos que combinan SVM y *Random Forest* con aprendizaje tipo *stacking*, sin que ninguno lograra superar de forma consistente el umbral objetivo del 90 % de *recall* en ambas clases. El SVM con kernel RBF resultó ser el modelo más estable y equilibrado para el alcance del proyecto.

Finalmente, la optimización del umbral de clasificación también requirió atención especial: el umbral estándar de 0.5 producía sesgo sistemático hacia una de las clases, por lo que fue necesario implementar la búsqueda exhaustiva basada en el *geometric recall* para encontrar un punto de operación más equilibrado.

## 8. Resultados

### 8.1. Evaluación sobre el conjunto de validación interno

Durante la fase de entrenamiento, el pipeline `StandardScaler`  $\rightarrow$  `PCA(30)`  $\rightarrow$  `SVC(RBF)` fue ajustado sobre 160 muestras del conjunto de entrenamiento. El subconjunto de validación interno (40 muestras) se utilizó exclusivamente para la búsqueda del umbral óptimo mediante *geometric recall*. Esta separación garantiza que el umbral seleccionado no esté sobreajustado a los datos de prueba final.

El proceso de búsqueda identificó un umbral óptimo que maximizó el equilibrio entre *recall* positivo y negativo sobre el conjunto de validación, reportando métricas superiores al umbral estándar de 0.5 en ambas clases.

### 8.2. Evaluación sobre datos de prueba independientes

La evaluación final se realizó sobre el conjunto de prueba (`test.csv`), compuesto por 60 muestras nunca antes vistas durante el proceso de entrenamiento ni de optimización del umbral (30 positivas y 30 negativas). Los resultados obtenidos se resumen en la Tabla 3.

Tabla 3: Métricas del modelo SVM sobre el conjunto de prueba (60 muestras)

| Métrica                                   | Valor   |
|-------------------------------------------|---------|
| Exactitud ( <i>accuracy</i> )             | 73.33 % |
| Precisión ( <i>precision</i> )            | 0.71    |
| Sensibilidad positiva ( <i>recall +</i> ) | 0.80    |
| Sensibilidad negativa ( <i>recall -</i> ) | 0.67    |
| F1-score                                  | 0.75    |
| Muestras evaluadas                        | 60      |

Matrices de Confusión — SVM VS RF VS SVM

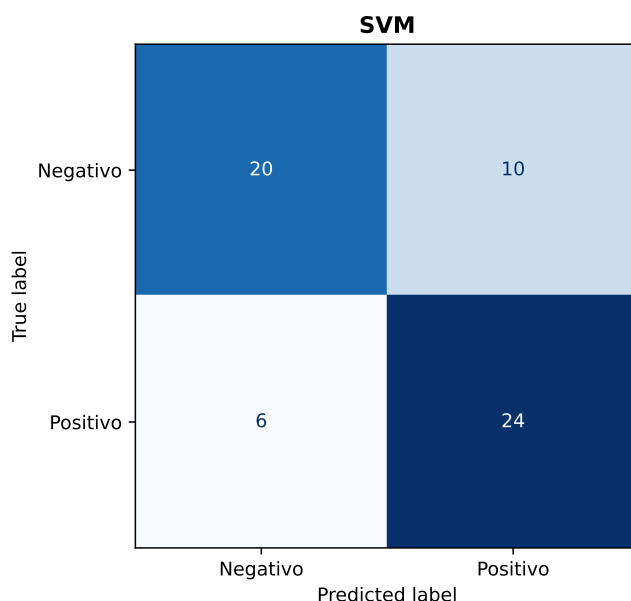


Figura 2: Matriz de confusión para el conjunto de prueba.

El significado de cada métrica en el contexto del problema se interpreta a continuación:

- **Exactitud (73.33 %)**: de las 60 muestras evaluadas, el modelo clasificó correctamente 44. Aunque es la métrica más intuitiva, puede ser engañosa cuando las consecuencias de los dos tipos de error son asimétricas.
- **Precisión (0.71)**: de cada 100 imágenes que el modelo predice como contaminadas, aproximadamente 71 lo están realmente. El 29 % restante corresponde a falsas alarmas (falsos positivos).
- **Recall positivo (0.80)**: el modelo detecta el 80 % de las imágenes realmente contaminadas. Esto implica que 1 de cada 5 contaminaciones reales no es detectada (falso negativo).
- **Recall negativo (0.67)**: el modelo identifica correctamente el 67 % de las imágenes limpias. El 33 % restante es clasificado erróneamente como contaminado.
- **F1-score (0.75)**: media armónica entre precisión y recall positivo. Resume el desempeño en la clase de interés (contaminación) en un único valor.

El modelo alcanzó una exactitud del **73.33 %** sobre los 60 datos de prueba. La matriz de confusión (Figura 2) confirma que el modelo comete **más falsos positivos que falsos negativos**: clasifica como contaminada una imagen limpia con mayor frecuencia que el caso contrario ( $\text{recall-} < \text{recall+}$ ). Este comportamiento es coherente con el uso de `class_weight='balanced'` y la optimización del umbral mediante *geometric recall*, cuyo objetivo es precisamente que ambas clases sean tratadas con igual penalización. En la práctica, el umbral óptimo encontrado tiende a reducir los falsos negativos a costa de aceptar más falsos positivos.

Desde la perspectiva de la aplicación, este sesgo es preferible en un contexto de control de calidad: es más costoso dejar pasar un producto contaminado que detener innecesariamente uno limpio. No obstante, el desempeño global (73.33 %) se encuentra por debajo del objetivo declarado del 90 % de *recall* en ambas clases, limitación atribuible principalmente al tamaño reducido del dataset y a la variabilidad no controlada en las condiciones de captura.

Además, se realizó una segunda prueba utilizando los 260 datos disponibles, incluyendo las muestras que el modelo ya había visto durante el entrenamiento. Los resultados obtenidos se presentan en la Tabla 3, y la matriz de confusión correspondiente puede consultarse en la Figura 3. En este caso, la exactitud aumenta a 89.5 %, lo cual es esperable, ya que la evaluación incorpora datos usados en el ajuste del modelo y, por tanto, no representa una prueba completamente independiente. Aun así, este resultado permite verificar que el modelo sí logró aprender patrones relevantes del conjunto completo de imágenes. Además, se mantiene la misma tendencia observada anteriormente: el modelo continúa generando más falsos positivos que falsos negativos, lo que confirma su comportamiento conservador hacia la detección de posibles contaminaciones.

Tabla 4: Métricas del modelo SVM sobre todo el conjunto de datos

| Modelo | Accuracy | Prec. | Rec+ | Rec- | F1   |
|--------|----------|-------|------|------|------|
| SVM    | 0.8950   | 0.88  | 0.91 | 0.88 | 0.90 |

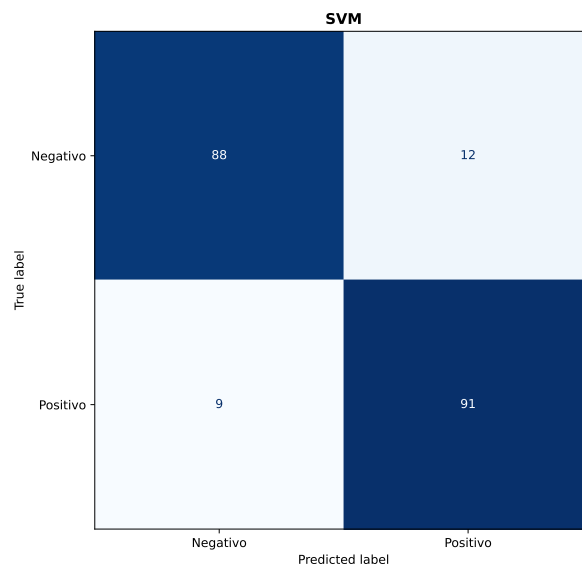


Figura 3: Matriz de confusión para el conjunto de prueba.

### 8.3. Análisis de generalización

La generalización de un modelo se mide por su capacidad de mantener un desempeño similar entre los datos de entrenamiento y los datos nunca antes vistos. La Tabla 5 compara el desempeño del modelo en ambos escenarios.

Tabla 5: Comparación de desempeño: datos vistos vs. datos nunca vistos

| Conjunto                       | Muestras | Accuracy | Rec+  | Rec-  |
|--------------------------------|----------|----------|-------|-------|
| Entrenamiento + prueba (todos) | 260      | 0.895    | 0.91  | 0.88  |
| Solo prueba (nunca vistos)     | 60       | 0.733    | 0.80  | 0.67  |
| <b>Diferencia</b>              | –        | –0,162   | –0,11 | –0,21 |

La caída de 89.5 % a 73.3 % entre el conjunto completo y el conjunto de prueba independiente indica un grado moderado de sobreajuste, esperable dado el reducido tamaño del dataset (200 muestras de entrenamiento). El *recall* negativo es el indicador más afectado (–0,21), lo que sugiere que el modelo tiene mayor dificultad para reconocer imágenes limpias bajo condiciones de captura no vistas durante el entrenamiento.

Aun así, la brecha de generalización no es crítica: el modelo mantiene un *recall* positivo del 80 % en datos nunca vistos, lo que significa que detecta 4 de cada 5 contaminaciones reales en condiciones nuevas. Para el alcance del proyecto y el tamaño del dataset disponible, este nivel de generalización se considera aceptable.

## 9. Conclusiones

El presente proyecto permitió desarrollar un sistema completo de clasificación binaria de imágenes para la detección de contaminación en una línea de producción simulada, abarcando desde la adquisición y preprocesamiento de las imágenes hasta el entrenamiento, optimización y evaluación del modelo seleccionado.

Se demostró que un pipeline basado en máquina de soporte vectorial con kernel RBF, precedido de estandarización y reducción de dimensionalidad mediante PCA, constituye una solución viable para el problema planteado. De entre los 7 algoritmos evaluados, incluyendo árboles de decisión, Naive Bayes, KNN, *Random Forest* y ensambles híbridos SVM-RF— SVM con kernel RBF ofreció el mejor equilibrio entre exactitud y *recall* en ambas clases, con resultados estables entre ejecuciones.

Sobre el conjunto de prueba independiente (60 muestras nunca vistas), el modelo alcanzó una exactitud del 73.33 %, con un comportamiento conservador que produce más falsos positivos que falsos negativos. Esta tendencia, inducida por el uso de `class_weight='balanced'` y la optimización del umbral mediante *geometric recall*, resulta adecuada para el contexto de inspección de calidad, donde es preferible generar una alarma innecesaria antes que omitir una contaminación real.

El desempeño obtenido se encuentra por debajo del objetivo declarado del 90 % de *recall* en ambas clases. Las principales causas identificadas son el tamaño reducido del dataset de entrenamiento (200 muestras) y la variabilidad no controlada en las condiciones de captura entre distintos dispositivos y sesiones. Ambos factores limitan la capacidad de generalización del modelo de forma independiente al algoritmo empleado.

Como trabajo futuro, se recomienda ampliar el dataset con imágenes capturadas bajo condiciones más homogéneas, explorar técnicas de aumento de datos (*data augmentation*) para compensar el tamaño reducido del conjunto de entrenamiento, y evaluar arquitecturas de redes neuronales convolucionales con mayor volumen de datos, las cuales podrían aprovechar mejor la estructura espacial de las imágenes que la representación vectorial plana utilizada en este trabajo.

## 10. Modelos fundacionales utilizados

El modelo fundacional utilizado en este proyecto fue Claude Code. Esta herramienta resultó de gran apoyo para mejorar las bases de código previamente desarrolladas para cada modelo, facilitando la revisión, optimización y organización de la implementación. Asimismo, fue útil en el proceso de documentación del código, permitiendo estructurar de forma más clara las funciones, procedimientos y componentes principales del proyecto.

## Referencias

- [1] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.
- [2] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [3] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Pearson, 2018.
- [4] G. Bradski, "The OpenCV library," *Dr. Dobb's Journal of Software Tools*, vol. 25, no. 11, pp. 120–125, 2000.
- [5] I. T. Jolliffe, *Principal Component Analysis*, 2nd ed. New York: Springer, 2002.
- [6] N. Otsu, "A threshold selection method from gray-level histograms," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, 1979.
- [7] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York: Springer, 2006.
- [8] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [9] J. C. Platt, "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods," in *Advances in Large Margin Classifiers*, A. J. Smola, P. Bartlett, B. Schölkopf, and D. Schuurmans, Eds. MIT Press, 1999, pp. 61–74.